Lesson 11 Teacher's Key: Wave After Wave

Godot Engine - Instructor Reference Only

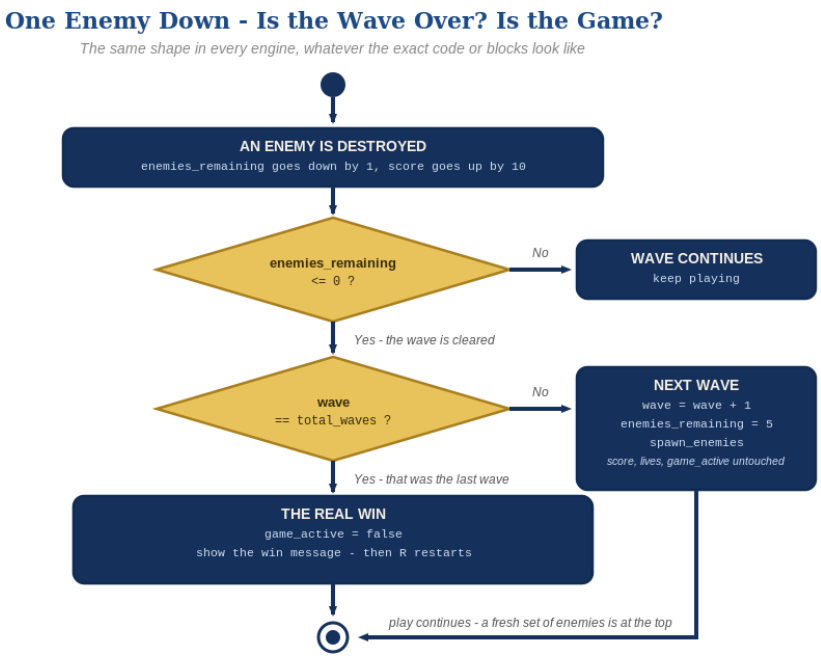
VIDEO GAME CREATION

What This Lesson Teaches

Reusing an existing, named function (Main's spawn_enemies) from a new caller - and a first look at why an engine sometimes needs you to wait before changing the scene (call_deferred), now in the middle of a collision rather than at load time.

Level 1 - Full Worked Solution

- Lesson 10 already extracted spawn_enemies() onto Main. This week it is called from a third place, the else branch of on_enemy_destroyed(), and its code does not change at all.
- wave and total_waves live on GameManager (Rule 13), alongside score and lives. update_hud() gets one line, and start_game() gets wave = 1.
- The new wave is requested with get_parent().call_deferred("spawn_enemies"). A wave is cleared inside a collision callback, and Godot refuses to add collision objects to the scene while it is still flushing collision queries (the Debugger shows a 'flushing queries' error). Deferring the call waits until the end of the frame, the same idea as last week's start_game.call_deferred().



Clearing a wave asks a second question. If it was not the last wave, the game spawns a fresh set of enemies and carries on. The same shape in every engine.

Why This Structure

- This mirrors Lesson 10's lesson exactly: one named routine, several callers. Now the caller is the game itself, not the player or the launch.
- The nested if/else keeps the win message reserved for the true end of the round: not every state change deserves the same visual weight.

Level 2 / Level 3 Talking Points

- Ask students to name every place spawn_enemies() is called from (start_game() and the else branch) and what triggers each: the first load or R, and the game itself.
- For students comparing engines: Godot's call_deferred is a deliberate wait; Scratch's broadcast and Unity's flag-and-Update achieve the same 'ask now, do it at a safe moment' idea.
- Teacher check: the deferral in the else branch is a precaution against Godot's collision-flush restriction. If a student's enemies are spawned from a signal fired inside a physics callback and no error appears for them, the deferral is still harmless and correct.
- The Stalemate (Level 3): do NOT hand over the fix. It was Lesson 10's Level 3 and is deliberately left in so it becomes irritating enough that students go back and fix it. Waves make it worse: after any escape, enemies_remaining can never reach 0, so that wave never ends - at any of the three waves. Confirm it is real if a student reports it, name it (the Stalemate), and nudge: 'what has to be true for a wave to be over?' The worked fix below makes an escape also resolve an enemy, and routes both the kill path and the escape path through ONE wave-cleared decision. Accept any design that guarantees

every wave ends in exactly one outcome. Test: 3 kills and 2 escapes, 4 and 1, 0 and 5 (the last should be a loss only if lives run out), in each of the three waves.

Level 2 - complete code:

```
# Level 2 is verification only - the Level 1 code is already correct, so
no lines change.
# Checks: (1) clear all five three times: waves 1 and 2 continue, wave 3
shows WinText;
# (2) lose the last life in wave 2: LoseText, no new wave;
# (3) R after a win and after a loss: wave 1, score 0, lives 3, exactly
five enemies.
```

Level 3 - complete code (additions on top of Level 1 highlighted):

Highlighted lines are the Level 3 additions on top of the Level 1 code. Methods shown are the final versions of everything Level 3 touches.

```
# Setup: add a Label node named WaveAnnouncementText near the middle of
the screen and untick Visible in the Inspector.
# GameManager's script - Level 3 (final versions of everything Level 3
touches):
func start_game():
    for old_enemy in get_tree().get_nodes_in_group("enemies"):
        old_enemy.queue_free()
    enemies_remaining = 5
    lives = 3
    score = 0
    wave = 1
    game_active = true
    get_node("../WinText").hide()
    get_node("../LoseText").hide()
    get_node("../WaveAnnouncementText").hide()
    update_hud()
    get_parent().spawn_enemies()

func on_enemy_destroyed():
    enemies_remaining -= 1
    score += 10
    update_hud()
    if enemies_remaining <= 0:
        wave_cleared()

func on_enemy_escaped():
    lives -= 1
    enemies_remaining -= 1
    update_hud()
    if lives <= 0:
        game_active = false
        get_node("../LoseText").text = "Game Over - Press R to Restart"
        get_node("../LoseText").show()
    elif enemies_remaining <= 0:
        wave_cleared()

func wave_cleared():
    if wave >= total_waves:
        game_active = false
        get_node("../WinText").text = "You Win! Press R to Restart"
        get_node("../WinText").show()
```

```
    else:
        wave += 1
        enemies_remaining = 5
        update_hud()
        get_parent().call_deferred("spawn_enemies")
        announce_wave()

func announce_wave():
    var banner = get_node("../WaveAnnouncementText")
    banner.text = "Wave " + str(wave) + "!"
    banner.show()
    await get_tree().create_timer(1.0).timeout
    banner.hide()
```

Common Student Mistakes

- Game-state drift: wave not reset to 1 in start_game(), so a restart after wave 3 begins at wave 3. Also a Level 3 announcement left visible after a restart.
- Calling spawn_enemies() directly in the else branch instead of deferring it: an error about flushing queries appears and no new wave spawns.
- Checking only enemies_remaining <= 0 without the wave comparison, so clearing wave 1 immediately wins.
- Spelling the deferred function name differently from the definition: call_deferred("spawn_enemies") fails silently into the Output panel.
- A student reports a wave that never ends: that is the Stalemate. Do not hand over the fix.